DATA ENGINEERING FOUNDATIONS / MODULE 02

# Mindset: EMR 101

Glue vs EMR vs Spark. What runs underneath. And the **30 PySpark functions** that do 90% of the work — with the code.


AUTHOR
**Surender Sara**

COMPANY
**NorthBay Solutions**



ISSUED
**August 2026**

FORMAT
**61 slides · 90 min**


Follows Module 01 — PySpark on AWS: Zero to Architecture

Applied. Bring the questions.

ORIENTATION

# Why a second session

### Module 01 answered *why*

Why distributed compute exists. HDFS, MapReduce, the shuffle, the S3 flip. Architecture from the ground up.

That was the **map of the territory**.

### Module 02 answers *how*

Which service to actually pick and what it costs. What is running underneath your Python. And the **specific functions** you will type every day.

This is the **vehicle**.

### The thesis of this session

If you know a **data warehouse**, if you know **OLAP**, if you know **SQL window functions** — you already know 90% of PySpark. You just don't know the spelling yet.

Today is mostly a **translation exercise**.

### The uncomfortable part.

An LLM will write correct PySpark syntax for you in three seconds. That is real, and it is not going away. So your value is **not** memorising boilerplate — it is knowing when the generated code will shuffle 400 GB, skew onto one core, or quietly cost $900. This session is about that judgement.

MAP

# Ten moves

- **01** Who built this, and what is running underneath
- **02** Is MapReduce still the core? Yes and no
- **03** Glue vs EMR vs EMR Serverless — with real money
- **04** Three kinds of DataFrame, and which to use
- **05** SQL → Pandas → PySpark: the translation table
- **06** **The 30 functions** — eight families, applied
- **07** The 1 TB question: how 64 cores produce one global answer
- **08** Iceberg and what really happens on write
- **09** The four knobs that actually matter
- **10** Reviewing code you did not write

> Section 06 is a third of the session and is the reason you are here. Everything before it is context; everything after it is consequence.

THE DATASET

# One table, all session

```
# glue_catalog.silver_db.transactions  (Apache Iceberg on S3)

tx_id           string
customer_id     string
region          string       # NA | EMEA | APAC
department      string       # Grocery | Electronics | ...
channel         string       # web | app | store
status          string       # completed | refunded | pending
amount          decimal(12,2)
currency        string
customer_email  string
tx_ts           timestamp
--- partitioned by ---
year, month, day
```

- Every code example in this deck runs against **this one table**. No toy word counts.
- It is **Iceberg**, on S3, registered in the **Glue Data Catalog** — the same table Athena, EMR and Glue all see.
- 1.4 billion rows. **~1 TB** on disk as Parquet.
- It is deliberately **dirty**: mixed-case regions, whitespace in emails, nulls in `channel`, refunds as negative and positive amounts.
- Everything you would actually hit on a Tuesday.

> **Write this schema down.** Sixty slides from now you will still be reading it.

~9 MIN

# 01

# What is actually running

Who wrote Spark, whose JVM executes it, and whether anyone is billing you for it.

PROVENANCE

# Who invented PySpark

### 2009 — UC Berkeley AMPLab

**Matei Zaharia**, as a PhD project. The goal was narrow: kill MapReduce's disk bottleneck for **iterative** workloads.

### 2013 — Apache + PySpark

Donated to the **Apache Software Foundation** as a top-level project. The **Python API** lands the same year. The creators found **Databricks**.

### Today

Apache Spark is **open source**. AWS, Microsoft, Google and Databricks package and operate the same engine as managed services.

**Not Oracle. Not Google. Not AWS.** Google published the papers that inspired Hadoop, but Spark is an independent open-source project. Glue and EMR are *wrappers* around software you could download and run on your laptop tonight for free.

**2009**
AMPLab prototype

**2013**
Apache + PySpark

**Apache 2.0**
the licence you operate under

**$0**
in engine licence fees

THE LICENCE QUESTION

# "If it's a JVM, isn't that Oracle?"

- **The specification** was written by Sun, then inherited by Oracle. Oracle owns the **Java trademark** and chairs the process.
- **The implementation you run is not Oracle's.** Sun open-sourced the HotSpot JVM in 2006–07 as **OpenJDK**, under GPLv2 **with Classpath Exception**.
- That exception is the whole point: you can run, ship and build commercial software on the JVM **without fees and without open-sourcing your code**.
- In 2019 Oracle put its *own branded build* behind a paid subscription. The entire cloud and big-data industry moved to OpenJDK builds instead.
- On AWS your Spark runs on **Amazon Corretto** — AWS's own OpenJDK distribution, free, patched by AWS, tuned for Graviton and x86.

### So, plainly

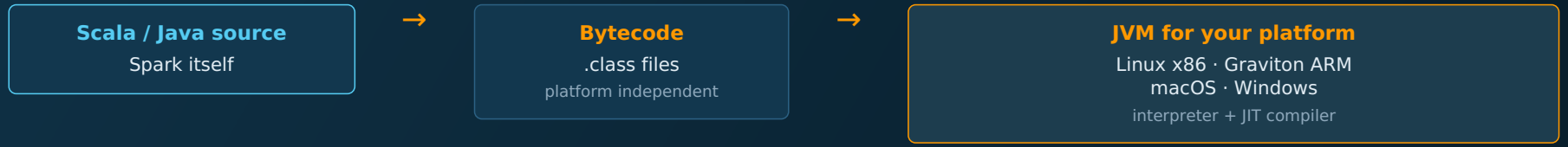
**No.** Oracle is not billing you for the JVM under your Glue job.

**Spec** Oracle stewards it
**Engine** OpenJDK, GPLv2+CPE
**On AWS** Amazon Corretto
**Alternatives** Eclipse Temurin, Red Hat

Worth knowing because this is the first question a procurement or security reviewer will ask you.

THE JVM

# Bytecode, and why it matters to you




## Write once, run anywhere — and why you care

The **same** Spark bytecode runs on your laptop, on a Graviton EMR node, and in a Glue container. That is why the job you tested locally behaves the same at 1 TB.

It is also why **Graviton is nearly free performance** on EMR: swap the instance family, change nothing in your code, take the ~20% price/performance win.

## The JIT is why Spark is fast

The JVM does not just interpret. It **watches which code paths are hot and compiles them to native machine instructions at runtime**.

Spark's Tungsten engine exploits this deliberately: it *generates* Java source for your query at runtime so the JIT can compile your specific `groupBy` into tight native code.

PY4J

# Python holds the wheel. The JVM drives.

**Your Python**

`df.groupBy(...)`

does zero row processing

→

**Py4J**

local socket

translates the call

→

**JVM — Spark Core**

Scala. Builds the DAG.
Catalyst. Tungsten.

this is where everything happens

## What crosses the bridge

**Instructions**, not data. "Add a filter on status." "Group by region." A few kilobytes of plan.

Your 1 TB never enters a Python process. Which is exactly why native DataFrame operations run at **full Scala speed**.

## The one exception

A **Python UDF** forces every row out of the JVM, through pickle, into a Python worker, and back.

1.4 billion round trips. This is the single largest self-inflicted wound in PySpark, and we come back to it in Section 06.

## Practical consequence

Stack traces will show you **Java exceptions** ( `Py4JJavaError` ). Read past the Python frames to the Scala cause — that is where the real error is.

Learn this now; it saves hours later.

~4 MIN

# 02

# Is MapReduce still the core?

Yes conceptually. Emphatically no architecturally. The distinction matters.

BOTH ANSWERS ARE RIGHT

# Conceptually yes. Architecturally no.

### YES — THE PATTERN SURVIVED

- **Map:** transform each record independently. That is `select`, `filter`, `withColumn`.
- **Reduce:** combine records across the cluster. That is `groupBy().agg()`, `join`, `distinct`.
- Map is cheap and local. Reduce moves data over the network. That has been true since 2004 and will be true in 2040.

### NO — THE ENGINE WAS REPLACED

| HADOOP MAPREDUCE | APACHE SPARK |
|---|---|
| Rigid 2-stage cycle: Map → Sort → Reduce | Arbitrary **DAG**: filter → join → map → aggregate → window |
| Intermediate results forced to **disk** after every phase | **Pipelined in RAM**; touches disk only on shuffle or spill |
| Fixed procedural code you wrote by hand | **Catalyst** rewrites your plan; **Tungsten** generates JVM bytecode |

**The line to give your students:** Spark kept MapReduce's mathematics and threw away its plumbing. If someone tells you "Spark is just MapReduce in memory," they have described one of the four things that changed.

~16 MIN

# 03

# Glue vs EMR

Same Spark. Same PySpark. Completely different bill and completely different job.

GLUE

# "AWS Glue" is three products in one name

### 1 · Glue ETL

THE ACTUAL SPARK ENGINE

You write real PySpark or Scala. AWS spins up an **ephemeral Spark cluster on containers**, runs your script, tears it down, bills per second.

**This is what engineers mean by "Glue".**

### 2 · Glue Data Catalog

THE METASTORE

Databases, tables, columns, partitions. A managed Hive metastore, shared by Athena, EMR, Redshift Spectrum and SageMaker.

Free for the first 1M objects and 1M requests a month.

### 3 · Glue DataBrew

THE NO-CODE TOOL

Point-and-click cleaning for analysts. Fill nulls, standardise dates, one-hot encode. **Billed separately.**

Nothing to do with your PySpark job.

### Clearing up "Glue has 250 transformations".

That number refers to the **visual recipe library** in DataBrew and Glue Studio — a no-code layer for analysts. It is not a Spark feature and it is not what you use. As an engineer you write PySpark and get Spark's ~400 built-in functions, not 250 recipes.

# What a DPU actually is

## The DPU, precisely

**1 DPU = 4 vCPU + 16 GB RAM.** That is the entire abstraction. It is Glue's way of selling you a container without saying "container".

Run a job on **10 DPUs** and Glue allocates **1 to the driver, 9 to executors**. Billed **per second**, one-minute minimum.

```
cost = DPUs × hours × rate

# 10 G.1X workers, 20 minutes, Standard
10 × (20/60) × $0.44 = $1.47 per run
× 30 runs/month      = ~$44
```

| WORKER | DPU | VCPU | RAM | REACH FOR IT WHEN |
|---|---|---|---|---|
| **G.1X** | 1 | 4 | 16 GB | Standard transforms and joins. **Start here.** |
| **G.2X** | 2 | 8 | 32 GB | Moderate transforms, ML, wider shuffles |
| **G.4X** | 4 | 16 | 64 GB | Large aggregations, big query plans |
| **G.8X** | 8 | 32 | 128 GB | Skewed aggregations, petabyte joins |
| **R.1X–R.8X** | — | — | 1:8 ratio | OOM but CPU is idle. Memory-heavy, ~$0.52/DPU-hr |

Hitting OOM? Move to an R-type before you scale G-types — you probably need RAM, not cores.

# Is Glue expensive? Yes. Use it anyway (usually).

## Glue is more expensive per unit of compute. Materially.

**Glue Standard:** $0.44 per DPU-hour — that is 4 vCPU + 16 GB.

**Glue Flex:** $0.29 per DPU-hour, ~34% off, for jobs that tolerate a delayed start. Nightly batch and backfills should almost always be on Flex.

**Equivalent raw EC2** (m5.xlarge: 4 vCPU, 16 GB): roughly $0.19 on-demand, and as low as **~$0.05 on spot**.

So you are paying something like a **2–8× premium** on raw compute.

## And it is usually still the right call

- **Zero idle cost.** A 3-minute job bills 3 minutes. An EMR cluster left up overnight bills all night for doing nothing.
- **Zero cluster ops.** No AMIs, no YARN tuning, no patching, no warm-up.
- **Zero staffing cost.** The engineer you didn't hire to babysit a cluster is more expensive than the premium.
- The premium only loses when compute runs **long and constantly** — which is exactly where EMR on EC2 with spot wins.

**The honest rule.** Under ~2 hours of compute a day, Glue's premium is noise and its operational savings are real. Over that, model it properly — the crossover arrives faster than people expect.

# DynamicFrame vs DataFrame

## DynamicFrame — Glue's own abstraction

A Glue-specific structure built for **messy, schema-drifting** data: nested JSON where a field is sometimes a string and sometimes an array.

It carries a **choice type** so a job doesn't die on a type conflict — it records both and lets you resolve later.

Also where Glue's bookmarks and native connectors live.

```
# Convert freely, at no cost
df  = dyf.toDF()                    # DynamicFrame → DataFrame
dyf = DynamicFrame.fromDF(df, ctx, "n")  # and back

# Typical real-world shape of a Glue job:
dyf = glueContext.create_dynamic_frame.from_catalog(...)
df  = dyf.toDF()                    # ◄ leave immediately

# ...all your real logic in native PySpark...

dyf_out = DynamicFrame.fromDF(result, ctx, "out")
```

**Advice:** ingest with a DynamicFrame if the source is genuinely messy, then `.toDF()` immediately and do everything else in native Spark. DynamicFrame has fewer functions, thinner docs, and no Catalyst optimisation of its own transforms.

# What an EMR cluster actually gives you

## EMR on EC2

THE FULL PLATFORM

**Primary** node runs YARN. **Core** nodes run tasks and hold HDFS/shuffle. **Task** nodes are compute only — put these on **spot**.

You pick instance families: c-series for CPU, r-series for memory, GPU for ML.

Full access to `spark-defaults.conf`, YARN,

GC flags, custom JARs, bootstrap actions, custom AMIs.

## Not just Spark

THE ECOSYSTEM

The same cluster can run **Spark, Trino/Presto, Hive, Flink, HBase, Zeppelin, JupyterHub**.

This is the actual reason large shops keep EMR: they need interactive Trino SQL and streaming Flink and batch Spark against one catalog and one set of nodes.

Glue gives you Spark, Python shell and Ray. That is the whole menu.

## The three deployment modes

PICK YOUR OPS BURDEN

**EMR on EC2** — you own the cluster. Cheapest per TB, highest ops.

**EMR Serverless** — submit a job, capacity appears. Custom vCPU/RAM ratios, open-source Spark, cheaper than Glue.

**EMR on EKS** — Spark as pods on your existing Kubernetes.

THE COMPARISON

# Same PySpark script. Three ways to run it.

Your code does not change. Provisioning, billing and blast radius do.

| DIMENSION | AWS GLUE ETL | EMR SERVERLESS | EMR ON EC2 |
|---|---|---|---|
| **Compute unit** | DPU — fixed tiers G.1X–G.8X, R.1X–R.8X | vCPU + memory + disk, chosen per worker | EC2 instances: m5, r5, c5, x2gd, GPU |
| **Cluster mgmt** | Fully hidden. Auto-scales DPUs. | Fully serverless, custom runtimes | You size primary/core/task and spot fleets |
| **Startup** | ~1–3 min (seconds with warm pools) | Seconds to ~1 min | 5–10 min for a full cluster |
| **Spark tuning** | Limited — AWS manages the runtime | Good — open-source Spark, most confs | Total. JVM, YARN, GC, JARs, AMIs. |
| **Max RAM/worker** | 128 GB (G.8X) | Flexible allocation | Terabytes (r5b, x2gd) — matters for huge shuffles |
| **Ecosystem** | Spark, Python shell, Ray | Spark, Hive | Spark, Trino, Flink, Hive, HBase, HUDI |
| **Cost profile** | Highest per hour, lowest ops | Middle — cheaper than Glue | **Lowest unit cost** with spot/RI, highest ops |
| **Reach for it** | Scheduled/event-driven batch ETL, catalog-native pipelines | Spiky Spark work needing custom sizing | Petabyte pipelines, 24/7 engines, interactive Trino |

APPLIED

# Three jobs, three answers

## Job A — nightly ingest

**12 MIN, 10 DPU, ONCE A DAY**

**Glue Standard** 10 × 0.2 × $0.44 = **$0.88**/run
**Glue Flex** = **$0.58**/run
**EMR on EC2** cluster spin-up alone is 5–10 min

**Glue Flex wins.** ~$17/month, zero ops. EMR's startup time is nearly as long as the job.

## Job B — 6-hour reprocess

**DAILY, ~200 CORES**

**Glue** 50 DPU × 6 × $0.44 = **$132**/run → ~$4,000/month
**EMR on EC2**, task nodes on spot → roughly **$25–40**/run

**EMR wins, and not narrowly.** At this duration the premium stops being noise.

## Job C — ad-hoc exploration

**AN ANALYST ASKING QUESTIONS**

Neither. Use **Athena** — SQL over the same Iceberg table, ~$5/TB scanned, no cluster, no job definition.

Spinning up Spark to answer `SELECT ... GROUP BY` is the most common waste of money in a young data platform.

**They are not rivals.** The mature pattern is Glue for discrete event-driven steps, EMR for the heavy crunching, Athena for exploration — all three pointed at **one Glue Data Catalog** and **one copy of the data in S3**.

Illustrative us-east-1 list prices. They move and vary by region — model your own job before committing.

~6 MIN

# 04

# Three kinds of frame

They look alike, they are not alike, and picking wrong costs you the cluster.

FRAMES

# Which DataFrame am I holding?

### 1 · pyspark.sql.DataFrame

**USE THIS — 95% OF THE TIME**

The distributed, lazy, Catalyst-optimised DataFrame. Method-chaining style, closer to SQL than to Pandas.

**Immutable.** No index. No guaranteed row order.

### 2 · pyspark.pandas.DataFrame

**FORMERLY KOALAS**

Real Pandas syntax — `.loc` , `.iloc` , `df['c'] = ...` — translated into Spark plans underneath.

Genuinely distributed. Useful for porting an existing Pandas script fast.

### 3 · pandas.DataFrame

**SINGLE MACHINE, IN THE DRIVER**

Plain Python Pandas. Lives entirely in **one process's RAM**.

You reach it only via `.toPandas()` , and only on data you have already aggregated down to something small.

```
# Converting is instant and costs nothing — it is the same plan
psdf = df.pandas_api()        # Spark DataFrame  → Pandas-on-Spark
df   = psdf.to_spark()        # Pandas-on-Spark  → Spark DataFrame
pdf  = df.limit(5000).toPandas()   # Spark → real Pandas. COLLECTS TO DRIVER.
```

# Same syntax, three different machines

| OPERATION | NATIVE PYSPARK | PANDAS-ON-SPARK | PLAIN PANDAS |
|---|---|---|---|
| Import | `from pyspark.sql import functions as F` | `import pyspark.pandas as ps` | `import pandas as pd` |
| Filter | `df.filter(F.col("amount") > 30)` | `psdf[psdf["amount"] > 30]` | `pdf[pdf["amount"] > 30]` |
| Add column | `df.withColumn("b", F.col("a")*2)` | `psdf["b"] = psdf["a"]*2` | `pdf["b"] = pdf["a"]*2` |
| Group & agg | `df.groupBy("dept").agg(F.avg("amt"))` | `psdf.groupby("dept")["amt"].mean()` | `pdf.groupby("dept")["amt"].mean()` |
| Sort | `df.orderBy(F.col("d").desc())` | `psdf.sort_values("d", ascending=False)` | `pdf.sort_values("d", ascending=False)` |

## Index

**Pandas** has a real sequential index. **Spark** has none — rows are scattered with no inherent order unless you evaluate an explicit `Window.orderBy()`. Pandas-on-Spark *fakes* one, and that fake can be expensive.

## Mutability

**Pandas** mutates in place. **Spark** is immutable: every operation returns a *new* DataFrame carrying an updated plan. Nothing in RAM is edited.

## Eager vs lazy

**Pandas** computes the moment you write the expression. **Spark** computes nothing until an action — which is why a broken job often fails on line 200, not line 12.

# Choosing, honestly

## When Pandas-on-Spark is the right call

- You have a working Pandas script and need it to run over 500 GB **this week**.
- Your team is analyst-heavy and the Spark API is a real adoption barrier.
- The logic is genuinely Pandas-shaped — lots of `.loc`, reshaping, index work.

## Why it is not the default

- It has to **manufacture an index**, which can add a shuffle you never asked for.
- Some Pandas semantics force **ordering guarantees** Spark otherwise wouldn't need.
- Not all of Pandas is implemented; you will hit gaps mid-project.
- Slower and harder to reason about in the Spark UI — the plan no longer resembles the code you wrote.

## And plain .toPandas()

Perfectly fine on a **result**: 40 rows of regional revenue for a chart.

Catastrophic on a **dataset**: it pulls every row into the driver's Python process. Always put a `.limit()` or an aggregation in front of it — every time, without thinking.

~5 MIN

# 05

# You already know this

Redshift, Snowflake, Oracle, Postgres, Pandas — it all maps.

TRANSLATION

# The mapping is nearly one to one

| WHAT YOU KNOW (SQL / OLAP) | PANDAS | PYSPARK | NOTE |
|---|---|---|---|
| `WHERE amount > 10` | `df[df.amount>10]` | `df.filter(F.col("amount")>10)` | identical semantics |
| `SELECT a, b` | `df[['a','b']]` | `df.select("a","b")` | in Spark this also prunes the file read |
| `GROUP BY d ... SUM()` | `df.groupby('d').sum()` | `df.groupBy("d").agg(F.sum("x"))` | triggers a shuffle |
| `OVER (PARTITION BY ... ORDER BY ...)` | `groupby().rolling()` | `Window.partitionBy().orderBy()` | **near-exact** translation |
| `CASE WHEN ... THEN ... ELSE` | `np.where(...)` | `F.when(...).otherwise(...)` | chainable |
| `DISTKEY / SORTKEY / CLUSTER BY` | (single machine — n/a) | `repartition() / partitionBy()` | same intent: co-locate related rows |
| `CREATE TABLE AS SELECT` | `df.to_parquet()` | `df.writeTo(...).create()` | Iceberg gives you real DDL |

**If you have written a Redshift window function, you have written a Spark window function.** The concepts you spent years learning — distribution keys, sort keys, partition pruning, predicate pushdown, cardinality — are the *same concepts*, with different spelling and a more honest error message.

TRANSLATION

# What's new, what's easier

## What genuinely is new

- **Laziness.** Nothing runs until an action. Errors surface late.
- **Partitions are visible.** In Redshift the optimiser hides distribution from you. In Spark it is your job.
- **The shuffle is yours to manage.** No cost-based optimiser will save you from a cartesian join on 1.4 billion rows.
- **Nothing is transactional by default.** That is why Iceberg exists.

## What is genuinely easier than SQL

- It is **Python**. Loops, functions, tests, modules, version control, imports.
- You can **break a 400-line query into named steps** and unit-test each one.
- Complex logic that would be a nested CTE nightmare is just… a function.
- `.explain()` shows you the real plan, free, any time.

## And you can still just write SQL

```
tx.createOrReplaceTempView("tx")

spark.sql("""
  SELECT region, SUM(amount) rev
  FROM tx WHERE year = 2026
  GROUP BY region
""")
```

Identical performance. Same Catalyst plan. Mix freely.

~28 MIN

# 06

# The 30 functions

Roughly 600 exist. Thirty do ninety percent of the work. Here they are, applied.

SECTION 06

# How many functions are there really?

**~400**
functions in pyspark.sql.functions

**~100**
DataFrame methods

**~80**
legacy RDD methods

**~150**
ML & streaming classes

> **~600–700 total. You will use about 30.**
>
> That is not laziness — it is the actual shape of the work. Data engineering is the same eight operations repeated against different schemas.

### The eight families

1. Shape & select
2. Filter & conditional
3. Strings
4. Dates & time
5. Aggregate
6. Join & reshape
7. Window analytics
8. Dedup, order, partition control

One slide each, applied to `silver_db.transactions` .

FAMILY 1 OF 8

# Shape & select

select · col · withColumn · withColumnRenamed · drop · cast · lit

```
from pyspark.sql import functions as F

tx = spark.read.table("glue_catalog.silver_db.transactions")

shaped = (tx
    .select("tx_id", "customer_id", "region", "department",
            "amount", "channel", "status", "tx_ts")   # 8 of 12 cols → column pruning at the file reader
    .withColumn("region",     F.upper(F.trim(F.col("region"))))     # overwrite in place, same name
    .withColumn("amount_usd", F.col("amount").cast("double"))       # decimal → double
    .withColumn("layer",      F.lit("silver"))                     # constant column
    .withColumnRenamed("tx_ts", "event_time")
    .drop("amount"))
```

## What each one buys you

**select** is not cosmetic — on Parquet/Iceberg it changes how many bytes leave S3.

**withColumn** with an existing name replaces that column. That is the normal way to clean a field.

**lit** wraps a Python value so Spark treats it as a column, not a variable.

## The gotcha

Forty chained `withColumn` calls build a forty-level nested plan and Catalyst gets slow analysing it.

Use `.withColumns({...})` (Spark 3.3+) or one `.select()` with all expressions when you have many.

FAMILY 2 OF 8

# Filter & conditional

filter/where · isin · between · when/otherwise · coalesce · fillna · dropna

```
clean = (shaped
    .filter(F.col("year") == 2026)                              # PARTITION FILTER → whole folders never listed
    .where(F.col("status").isin("completed", "refunded"))       # where == filter, pick one and be consistent
    .filter(F.col("amount_usd").between(0.01, 100000))
    .filter((F.col("region") == "NA") | (F.col("channel") == "app"))  # & | ~  with parentheses. NOT and/or/not.
    .withColumn("channel", F.coalesce(F.col("channel"), F.lit("unknown")))
    .withColumn("kind", F.when(F.col("amount_usd") < 0,    "refund")
                         .when(F.col("amount_usd") > 1000, "high_value")
                         .otherwise("standard"))
    .na.fill({"currency": "USD"})
    .na.drop(subset=["customer_id"]))
```

## Push filters up, always

Filter **before** joins, aggregations and windows. Catalyst usually does it for you — but it cannot push through a Python UDF.

The partition filter goes **first**: it is the only filter that prevents I/O rather than discarding rows you already paid to read.

## Four traps

1. `and / or / not` raise errors — use `& | ~` with brackets.
2. `col != None` is always false — use `isNull()` .
3. `filter` and `where` are the same method.
4. `coalesce` the function (null fallback) is unrelated to `coalesce` the method (partition merge).

FAMILY 3 OF 8

# Strings

lower · trim · substring · split · regexp_replace · concat_ws

```
# Real cleaning on a real dirty column
enriched = (clean
  .withColumn("email",
      F.lower(F.trim(F.col("customer_email"))))
  .withColumn("domain",
      F.split(F.col("email"), "@").getItem(1))
  .withColumn("dept_code",
      F.upper(F.substring("department", 1, 4)))
  .withColumn("sku_clean",
      F.regexp_replace("sku", "[^A-Za-z0-9]", ""))
  .withColumn("label",
      F.concat_ws(" | ", "region", "department", "channel")))
```

## Why this is the family people get wrong

String cleaning is where juniors reach for a Python UDF, because "it's just a regex".

Spark has **~90 string functions**, all compiled to JVM bytecode. A Python UDF doing the same regex is **10–100× slower** and blocks Catalyst from optimising anything around it.

## Worth knowing

**concat_ws** skips nulls; **concat** returns null if *any* input is null. That surprises everyone once.

**split** returns an array — use `.getItem(n)` or `[n]` .

**regexp_extract** is the sibling you'll want for parsing structured strings.

FAMILY 4 OF 8

# Dates

to_date · hour · date_trunc · datediff · date_add · add_months · current_date

```
dated = (enriched
  .withColumn("tx_date",  F.to_date("event_time"))
  .withColumn("tx_hour",  F.hour("event_time"))
  .withColumn("week",     F.date_trunc("week", "event_time"))
  .withColumn("days_ago",
      F.datediff(F.current_date(), F.col("tx_date")))
  .withColumn("refund_by",
      F.date_add(F.col("tx_date"), 30))
  .filter(F.col("tx_date") >=
          F.add_months(F.current_date(), -3)))
```

```
# set this once, at the top, in every job
spark.conf.set("spark.sql.session.timeZone", "UTC")
```

## The timezone bug you will absolutely ship

Spark's session timezone defaults to the **JVM's** timezone. Your laptop is not UTC. The Glue container might be. The EMR node might be something else entirely.

Result: `to_date()` shifts by hours, rows land in the wrong `day=` partition, and your daily counts are quietly wrong at the boundaries. Nothing errors. Nothing alerts.

**Set it explicitly to UTC in every job.** Convert to local time only at the presentation layer.

**date_trunc** is your friend for any "by week / by month" roll-up — it is the direct equivalent of `DATE_TRUNC` in Redshift and Snowflake.

FAMILY 5 OF 8

# Aggregate

groupBy · agg · count · sum · avg · min/max · countDistinct · approx_count_distinct · collect_set

```
daily = (dated
    .groupBy("region", "department", "tx_date")                          # ◄◄ SHUFFLE HAPPENS HERE
    .agg(
        F.count("*").alias("tx_count"),                    # counts rows, nulls included
        F.count("channel").alias("with_channel"),          # counts NON-NULL only — a different number
        F.sum("amount_usd").alias("revenue"),
        F.avg("amount_usd").alias("avg_ticket"),
        F.min("amount_usd").alias("min_amt"), F.max("amount_usd").alias("max_amt"),
        F.countDistinct("customer_id").alias("customers"),           # EXACT. expensive. see section 07.
        F.approx_count_distinct("customer_id", 0.02).alias("cust_est"),  # HyperLogLog, 2% error, ~free
        F.collect_set("channel").alias("channels_used"),
    ))
```

## count("*") vs count(col)

The first counts **rows**; the second counts **non-null values**. The difference between them is often exactly the data-quality metric you wanted.

## countDistinct is the expensive one

Sums and counts get **partially aggregated before the shuffle**. An exact distinct cannot — it must see every value. Use **approx_count_distinct** for dashboards: 2% error, a fraction of the cost.

## collect_set will OOM you

It builds an **unbounded array in one executor's memory** per group. One hot key and that array is 400 million elements. Cap it, or don't use it.

FAMILY 6 OF 8

# Join & reshape

join · broadcast · left_anti · left_semi · unionByName · explode · pivot

```
customers = spark.read.table("glue_catalog.silver_db.dim_customer")   # ~40 MB

joined  = daily.join(F.broadcast(customers), on="customer_id", how="left")  # small side shipped everywhere: NO shuffle
orphans = daily.join(customers, "customer_id", "left_anti")                  # rows with NO match → the QA workhorse
matched = daily.join(customers, "customer_id", "left_semi")                  # rows WITH a match, no extra columns

stacked = jan.unionByName(feb, allowMissingColumns=True)                    # by NAME. never use union() — it is positional.

items   = orders.withColumn("item", F.explode("line_items")) \              # 1 order row → N item rows
                .select("order_id", "item.sku", "item.qty", "item.price")

matrix  = daily.groupBy("region") \
               .pivot("channel", ["web", "app", "store"]) \                       # ALWAYS pass the value list
               .agg(F.sum("revenue"))
```

## broadcast is the highest-leverage word in PySpark

If one side fits in executor memory (<100 MB), broadcasting it **removes the shuffle entirely** — routinely 5–20× for one word. AQE often does it now; be explicit anyway.

## Three joins nobody teaches you

**left_anti** — "which transactions have no customer record?" The best data-quality tool you own. **left_semi** — filter by existence without dragging in columns. **unionByName** — because `union()` matches by *position* and will happily put emails in the amount column.

FAMILY 7 OF 8

# Windows

Window.partitionBy/orderBy · row_number · rank · dense_rank · lag · lead · rowsBetween

```
from pyspark.sql.window import Window

w_rank = Window.partitionBy("region").orderBy(F.col("revenue").desc())
w_run  = (Window.partitionBy("customer_id").orderBy("tx_date")
                .rowsBetween(Window.unboundedPreceding, Window.currentRow))

top10 = (daily
    .withColumn("rn",        F.row_number().over(w_rank))        # 1, 2, 3, 4  — no ties
    .withColumn("rnk",       F.rank().over(w_rank))              # 1, 2, 2, 4  — gaps
    .withColumn("dense",     F.dense_rank().over(w_rank))        # 1, 2, 2, 3  — no gaps
    .withColumn("prev_rev",  F.lag("revenue",  1).over(w_rank))
    .withColumn("next_rev",  F.lead("revenue", 1).over(w_rank))
    .withColumn("lifetime",  F.sum("revenue").over(w_run))        # running total per customer
    .filter(F.col("rn") <= 10))                                 # top 10 departments per region
```

## This is your Redshift knowledge, unchanged

`OVER (PARTITION BY region ORDER BY revenue DESC)` becomes `Window.partitionBy("region").orderBy(desc)` .

Same semantics, same frame clauses, same rank/dense_rank distinction. If you can do it in a warehouse you can do it here today.

## The mistake that halves your cluster

A window with **no** `partitionBy` means **every row must reach one executor** to establish global order.

319 cores idle, one core doing 1.4 billion rows. Always partition a window unless you truly need a global sequence — and if you do, question the requirement.

FAMILY 8 OF 8

# Dedup, order, partitions

distinct · dropDuplicates · orderBy · sortWithinPartitions · repartition · coalesce · cache

```
# --- dedup ---
tx.distinct()                          # all columns
tx.dropDuplicates(["tx_id"])           # keeps an ARBITRARY row

# deterministic dedup — what you actually want
w = Window.partitionBy("tx_id") \
          .orderBy(F.col("ingested_at").desc())
latest = (tx.withColumn("r", F.row_number().over(w))
            .filter("r = 1").drop("r"))

# --- order ---
df.orderBy(F.col("revenue").desc())     # GLOBAL → range shuffle
df.sortWithinPartitions("tx_date")   # local only → nearly free

# --- partition control ---
df.repartition(400, "region")          # full shuffle, exact count
df.coalesce(40)                      # merge only, no shuffle
df.cache(); df.unpersist()           # only if reused 3+ times
```

## repartition vs coalesce — know this cold

**repartition(n)** does a full shuffle. It can **increase** partitions and **rebalance skew**. Expensive but sometimes exactly right.

**coalesce(n)** only **merges** existing partitions. No shuffle, so it is cheap — but it can only reduce, and it can leave you with uneven partitions.

**Use coalesce before a write** to avoid emitting 2,000 tiny files. Use repartition when the data itself is unbalanced.

## orderBy is not free

A global sort forces a **range-partitioning shuffle** across the whole dataset (Section 07 shows how).

Never sort just to make output "look nice" before writing — the consumer will re-sort anyway and you have paid a full shuffle for aesthetics.

SECTION 06

# The cheat sheet

Photograph this one too. Pin it above your desk for a month.

## Shape

`select` `col` `withColumn` `withColumnRenamed` `drop` `cast` `lit` `alias`

## Filter

`filter/where` `isin` `between` `when/otherwise` `coalesce` `fillna` `dropna` `isNull`

## Strings & dates

`lower` `trim` `split` `regexp_replace` `concat_ws` `to_date` `date_trunc` `datediff`

## Aggregate

`groupBy` `agg` `count` `sum` `avg` `min/max` `countDistinct` `approx_count_distinct`

## Join & reshape

`join` `broadcast` `left_anti` `left_semi` `unionByName` `explode` `pivot`

## Window & control

`Window` `row_number` `rank` `lag/lead` `dropDuplicates` `orderBy` `repartition` `coalesce`

**That is the list. Roughly forty names; thirty you'll type weekly.**

Everything else you look up when the need arrives — and by then you know what to search for, which is the actual skill.

APPLIED

# Twenty of the thirty, in one real job

Nothing exotic. This is what production PySpark actually looks like.

```
# Business question: top 10 departments per region by revenue, last 90 days,
# with week-over-week movement and the customer's lifetime spend. One job.

base = (spark.read.table("glue_catalog.silver_db.transactions")
    .filter(F.col("year") == 2026)                                  # 1. partition pruning FIRST
    .select("tx_id","customer_id","region","department","amount","status","tx_ts")  # 2. column pruning
    .filter(F.col("status") == "completed")                          # 3. narrow before you shuffle
    .withColumn("region",  F.upper(F.trim("region")))
    .withColumn("week",    F.date_trunc("week", "tx_ts"))
    .filter(F.col("tx_ts") >= F.add_months(F.current_date(), -3)))

weekly = base.groupBy("region","department","week").agg(                    # 4. the one shuffle you meant to do
    F.sum("amount").alias("revenue"),
    F.approx_count_distinct("customer_id").alias("customers"))

w = Window.partitionBy("region","week").orderBy(F.col("revenue").desc())
final = (weekly.withColumn("rn", F.row_number().over(w))                       # 5. reuses the same partitioning
               .withColumn("wow", F.col("revenue") - F.lag("revenue").over(w))
               .filter("rn <= 10").coalesce(8))                          # 6. small result → few files
```

SECTION 06

# Five things you'll be tempted to write. Don't.

## 1 · The Python UDF

THE EXPENSIVE HABIT

Every row leaves the JVM. 10–100× slower and Catalyst goes blind.

**Instead:** search the ~400 built-ins first. Then a pandas/Arrow UDF. A plain Python UDF is a last resort you should be able to justify.

## 2 · .collect() / .toPandas()

THE DRIVER KILLER

Pulls the whole dataset into one Python process.

**Instead:** `show()` , `limit().toPandas()` , or write to S3 and query it.

## 3 · A loop over dates

THE ACCIDENTAL 365-JOB PIPELINE

```
for d in dates:
    df.filter(...).write...
```

**Instead:** one job, `partitionBy("date")` on write. Spark parallelises what your loop serialised.

## 4 · count() as a debug print

Every `df.count()` is a **full action** that re-executes the entire lineage above it. Five of them sprinkled through a script is five full passes over 1 TB.

**Instead:** `.cache()` if you genuinely need repeated counts, or count once at the end.

## 5 · orderBy before write

A global sort is a full range shuffle. If you are sorting so the output files "look tidy", you have paid a shuffle for nothing.

**Instead:** `sortWithinPartitions()` if you want local clustering, or nothing at all.

07

# The 1 TB question

~14 MIN

Sixty-four cores. A terabyte that fits in none of them. One correct global answer.

SECTION 07

# How does a partial answer become the final answer?

### The question, as a sharp student will ask it.

"I have a four-node EMR cluster, 64 cores total. My table is 1 TB. I want `SUM(amount) GROUP BY department`, and then sorted. The whole dataset has to be scanned for that to be correct. But if each core only ever sees a 128 MB slice, every sort and every subtotal it computes is **intermediate**, not final. So how does the cluster ever converge on the *real* answer without one machine eventually holding everything?"

### This is the right question

It is the question that separates people who use Spark from people who understand it. The answer is not "magic" and it is not "more RAM".

### The five-word answer

**Shrink first, then route by key.**

Once every record for a key is guaranteed to be on one machine, that machine's local answer *is* the global answer.

STEP 1

# Waves — the data streams past the cores

**1 TB Iceberg table**
on S3

→

**Driver plans splits**
reads Iceberg manifests
target ~128 MB each

→

**~8,000 partitions**
1,024,000 MB ÷ 128 MB

- You have **64 cores**, not 8,000. So Spark runs the tasks in **~125 sequential waves**.
- A core takes chunk #1, processes it, **drops the raw rows from memory**, takes chunk #2. It never holds more than its own slice.
- This is why 1 TB does not need 1 TB of RAM. It needs **64 × 128 MB ≈ 8 GB** of live working set at any instant.
- Nothing is read twice. There is no back-and-forth scanning. It is one **strictly forward, pipelined pass**.

## The arithmetic to put on the board

1 TB = 1,024,000 MB
÷ 128 MB = **8,000 tasks**
÷ 64 cores = **125 waves**

Live memory at any moment:
64 × 128 MB = **~8 GB**

Not 1 TB. Eight gigabytes.

STEP 2

# Map-side partial aggregation — the shrinker

### What each core does with its 128 MB

It does **not** ship raw rows anywhere. It maintains a tiny in-memory hash map:

```
{ "Grocery":     running_sum,
  "Electronics": running_sum,
  "Apparel":     running_sum }
```

128 MB might be **1,000,000 rows** across only **50 departments**. That core emits **50 subtotal rows**, not a million.

**1 TB**
raw input

**~200 MB**
intermediate subtotals after map-side combine

**5000x**
smaller before it touches the network

**This is the whole trick.** The shuffle is expensive, so Spark makes sure the thing being shuffled is *tiny*. By the time all 8,000 chunks are scanned, a terabyte of transactions has collapsed into a few hundred megabytes of partial sums.

STEP 3

# The hash shuffle — routing by key

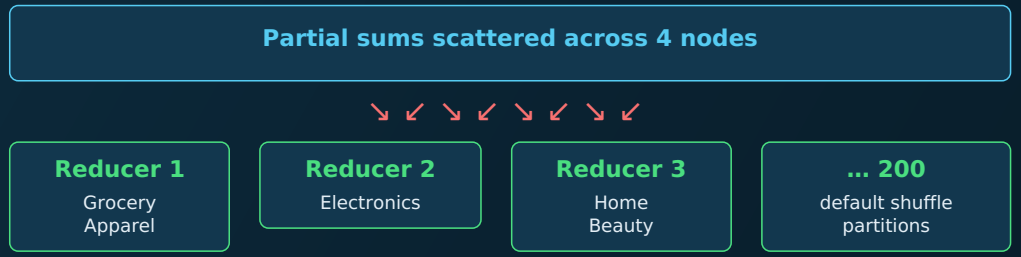
## The routing rule

Now thousands of partial subtotals sit scattered across four nodes. To get a true global sum, every subtotal for a department must **meet**.

Spark uses a deterministic hash:

```
target = hash(department)
         % spark.sql.shuffle.partitions
```

Every `"Grocery"` subtotal in the cluster — no matter which of the 8,000 tasks produced it — computes the **same** target. So they all land together.




One coordinated network exchange. Not a scan, not a search — a **deterministic address calculation**. Every record knows exactly where it must go, so nobody has to look for anything.

STEP 4

# Final global aggregation

## Why this is now the final answer

Reducer 2 owns **100% of the Electronics subtotals that exist anywhere in the 1 TB**. Not most. All.

So when it adds them together, that sum is not intermediate. It is **the true global answer**, computed on a machine that never held more than a few megabytes.

No coordination round. No second pass. No central assembler.

## The line for your students

**Correctness comes from the routing guarantee, not from anyone seeing all the data.**

You never needed a machine that could hold 1 TB. You needed a rule that guarantees all of a key's records reach the same place. That is the entire insight behind twenty years of distributed computing.

## What it cost

**One** pass over S3.
**One** network exchange of ~200 MB.
**Zero** full-dataset materialisation.

On 64 cores that is minutes, not hours — and the number is exact.

This is also why **skew hurts so much**. If 60% of your rows are `region='NA'`, one reducer receives 60% of the shuffled data. 199 reducers finish in seconds; that one runs for forty minutes while everything waits. The routing rule that guarantees correctness is the same rule that concentrates the pain.

STEP 5

# Global sorting — range partitioning

- "Fine for sums — but a **global ORDER BY** is different. Each machine only sees its own slice, so how can the whole thing be sorted?"
- Spark does not sort 1 TB in one place. It uses **range partitioning**.
- **1. Sample.** Spark takes a fast statistical sample of the data to estimate the distribution and pick boundary values.
- **2. Route by range**, not by hash. Each reducer receives one contiguous value band.
- **3. Local sort.** Each reducer sorts its own band in memory, spilling to disk if needed.
- **4. Concatenate.** Because reducer 1's minimum is guaranteed above reducer 2's maximum, the parts stitch together already in order.


**Globally sorted, with no global sort.** The dataset is ordered across the entire cluster and no single machine ever held more than one band.

This is why Spark runs a small extra job before a big `orderBy` — that job is the sampling pass. Now you know what it is when you see it in the UI.

STEP 6

# Where the shrink fails — and what to do

## Aggregates that shrink (combinable)

`sum` `count` `min` `max` `avg`

**avg** works because Spark shuffles *(sum, count)* pairs and divides at the end — it never averages averages.

These collapse 1,000,000 rows to 50 before the network. This is the good case.

## Aggregates that do not shrink

`countDistinct` `collect_list` `collect_set` `percentile` `median`

An exact distinct must see every value, so **every raw value crosses the network**. Your 5000× shrink becomes 1×.

This is why one `countDistinct` can double a job's runtime.

## What to do about it

**approx_count_distinct** uses HyperLogLog sketches, which *are* combinable. ~2% error, a fraction of the cost.

**percentile_approx** is the same idea for quantiles.

Ask whether the dashboard genuinely needs an exact number, or just a correct-looking one. Usually it is the latter.

## Lecture takeaway for the board.

**1.** Waves — cores stream slices; raw rows are discarded immediately. **2.** Map-side combine — a terabyte becomes megabytes. **3.** Hash routing — all records for key X reach the same worker, so its local answer is the global one. **4.** Range routing — sorted bands stitch together in order.

~11 MIN

# 08

# Iceberg & the write

What actually happens between *.write* and the table being visible.

# Why a table format exists at all

## The problem Iceberg was built to fix

S3 is an **object store**. There are no directories and there is **no atomic rename**.

Classic Hive-style Spark writes to a `_temporary/` folder, then renames on success. On a real filesystem a rename is a metadata flick. On S3 it is a **full byte-for-byte copy**.

Writing 200 GB means copying 200 GB a second time. And if the job dies mid-rename, readers see a half-written table.

## What Iceberg changes

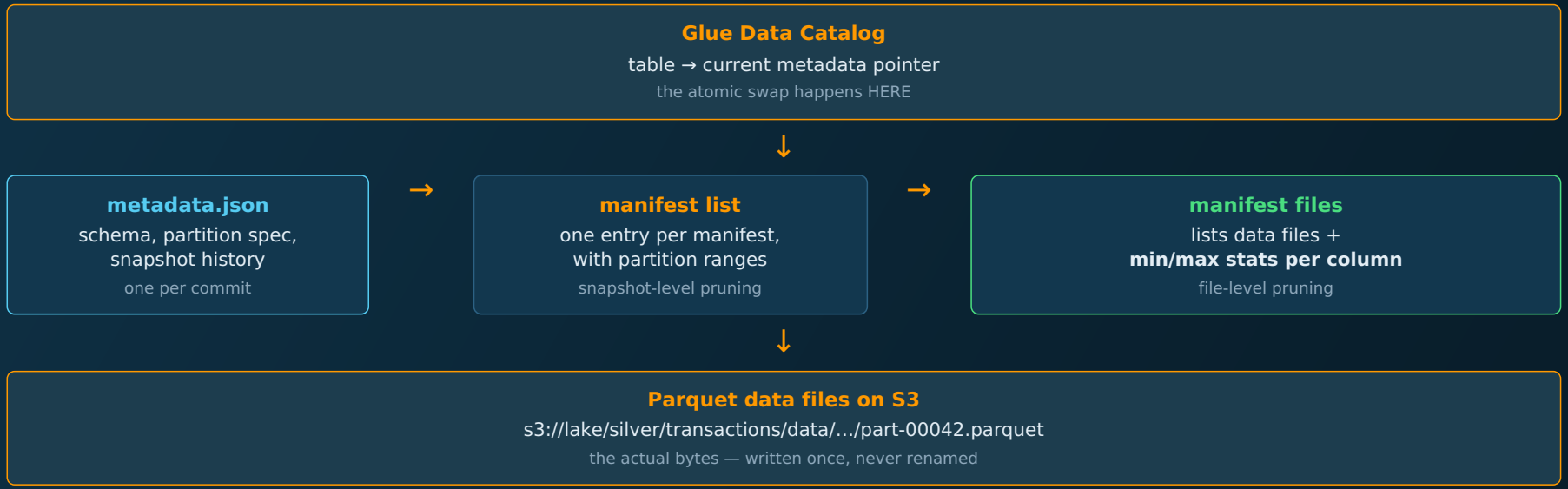
- Data files are written **directly to their final S3 location**. No temp folder, no rename, no copy.
- Visibility is controlled by a **metadata pointer**, swapped atomically at the end.
- Readers see the old snapshot until the instant of commit, then the new one. **Never a partial state.**
- Two writers can work concurrently and the commit is arbitrated safely.

## Why this is the industry default now

Iceberg is first-class in **Glue, EMR, Athena, Redshift and SageMaker**, and the Glue Data Catalog speaks it natively.

Delta Lake and Hudi solve the same problem; on AWS in 2026, Iceberg is the one to learn first.

# The anatomy




**Read this bottom-up to understand queries, top-down to understand writes.** A query walks *down* the tree, discarding whole manifests and whole files using min/max statistics before a single byte of Parquet is opened. A write walks *up*: data files first, manifest second, pointer last.

# The commit, step by step

## Phase 1 — Executors write

Each executor writes its output partitions as Parquet, **straight to the permanent S3 path**.

At this moment the files exist but the table does not know about them. No reader can see them. Nothing is broken if the job dies here — you just have orphan files to clean up.

## Phase 2 — Driver commits

The driver gathers every file path plus per-column **min/max statistics** from the workers.

It writes a new **manifest** (Avro) and a new **metadata.json** describing the new snapshot.

Then it performs one **atomic pointer swap** in the Glue Catalog.

## The instant of visibility

Before the swap: readers see the old snapshot, complete and correct.

After the swap: readers see the new one, complete and correct.

**There is no in-between.** That is what ACID means here, and it is why analysts stop seeing half-loaded tables at 3 a.m.

## Why it is also faster

No `_temporary` directory. No rename-as-copy. No expensive S3 **LIST** to discover what exists — the manifest already says.

On a wide Hive table, listing partitions can take longer than reading the data. Iceberg deletes that entire class of problem.

## And why time travel is nearly free

Each commit writes a **new** metadata.json and keeps the old ones. The data files are immutable and shared.

"Show me this table as of last Tuesday" is just: read an older pointer. No backup, no restore, no copy.

# The operations you'll actually run

```
# Write — create, append, or replace just the touched partitions
(final.writeTo("glue_catalog.gold_db.weekly_dept_revenue")
      .tableProperty("write.format.default", "parquet")
      .partitionedBy("region")
      .createOrReplace())

final.writeTo("glue_catalog.gold_db.weekly_dept_revenue").append()
final.writeTo("glue_catalog.gold_db.weekly_dept_revenue").overwritePartitions()  # idempotent reruns

-- Upsert: the thing a plain Parquet lake simply cannot do
MERGE INTO gold_db.customer_360 t USING updates s ON t.customer_id = s.customer_id
  WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *;

DELETE FROM silver_db.transactions WHERE customer_id = 'c-8842';      -- GDPR, one line
SELECT * FROM silver_db.transactions FOR TIMESTAMP AS OF '2026-08-01 00:00:00';

CALL glue_catalog.system.rewrite_data_files(table => 'silver_db.transactions');  -- compaction
CALL glue_catalog.system.expire_snapshots(table => 'silver_db.transactions',
                                  older_than => TIMESTAMP '2026-08-01');      -- housekeeping
```

**overwritePartitions() is the one to remember.** It replaces only the partitions your DataFrame actually contains. That makes a rerun **idempotent** — run it five times, get the same table — which is the property that lets you sleep during a backfill.

# What breaks if you ignore maintenance

## Housekeeping is not optional

- **rewrite_data_files** — compaction. Streaming or frequent appends produce thousands of small files; this merges them back toward ~128 MB.
- **expire_snapshots** — old snapshots pin old data files in S3. Without expiry your storage bill grows forever.
- **rewrite_manifests** — keeps metadata pruning fast on very wide tables.
- Glue can run **managed Iceberg compaction** for you, billed at the usual DPU rate (AWS's example: 30 min on 2 DPUs ≈ $0.44).

## The partition mistake to avoid

Iceberg supports **hidden partitioning**: declare `days(tx_ts)` and queries filtering on `tx_ts` get pruning automatically — no `WHERE day=` ritual, no derived columns leaking into your schema.

And it supports **partition evolution**: change the scheme later without rewriting history. Hive could never do this, which is why so many lakes are stuck with a partitioning decision someone made in 2019.

**Iceberg does not make bad layout fast.** It gives you transactions, evolution and cheap metadata — it does not rescue a table partitioned on `customer_id` with 900 million values. Layout decisions still matter exactly as much as they did in Module 01.

~7 MIN

# 09

# Four knobs

Out of hundreds of Spark configurations, these are the ones you will actually touch.

TUNING

# The four that matter

| CONFIGURATION | DEFAULT | WHAT IT CONTROLS | WHEN TO CHANGE IT |
|---|---|---|---|
| `spark.sql.files.maxPartitionBytes` | 128 MB | How S3 files are cut into **input** partitions on read | Rarely. Raise it if you have huge files and idle cores; lower it for very heavy per-row work. |
| `spark.sql.shuffle.partitions` | **200** | How many partitions come *out* of every shuffle | **Often.** 200 was chosen for 2015 laptops. On 320 cores you want 2–4× the core count. |
| `spark.sql.adaptive.enabled` (AQE) | true (Spark 3.x) | Re-optimises at runtime using real statistics: coalesces small shuffle partitions, splits skewed ones, converts joins to broadcast | Leave it on. Verify it is on. It fixes many of the problems people still hand-tune. |
| `spark.sql.autoBroadcastJoinThreshold` | 10 MB | Below this estimated size, the small side of a join is broadcast automatically | Raise to 50–200 MB when your dimension tables are bigger than 10 MB and executors have RAM to spare. |

**The 200 default is the single most common misconfiguration in production Spark.**

A 1 TB aggregation shuffling into 200 partitions gives you 200 tasks of ~1 GB each. With 320 cores, **120 of them sit idle** while the rest chew oversized partitions and spill to disk. Setting this correctly is frequently a 2–3× speedup for one line.

TUNING

# When the job is slow

## Diagnose in this order

- **1. Am I reading too much?** Check `PartitionFilters` and `PushedFilters` in `.explain()`. This is the cheapest possible win.
- **2. How many shuffles?** Count stages in the UI. Can one be broadcast away?
- **3. Is it skewed?** Compare **max vs median** task duration on the slowest stage. 40× means skew, not slowness.
- **4. Partitions vs cores?** Tasks < cores means idle hardware.
- **5. Any Python UDFs?** Look for `BatchEvalPython` in the physical plan.
- **6. Spilling?** Non-zero *spill (disk)* in the stage means partitions are too big for executor memory.

## Fixes, in the order you should try them

- Add the partition filter you forgot
- Set `shuffle.partitions` to 2–4× core count
- `broadcast()` the small side of the join
- Confirm **AQE** is enabled — it handles most skew automatically now
- If skew persists: **salt** the key, aggregate, then re-aggregate
- `coalesce()` before write to stop emitting small files
- Move to an **R-type** worker if you are OOM but CPU is idle

**Notice what is not on either list:** buying a bigger cluster. That is the last resort and it is usually the most expensive way to hide a layout problem.

~6 MIN

# 10

# Reviewing code you didn't write

Because increasingly, you didn't.

# A six-point review for generated PySpark

## The honest situation

PySpark is highly standardised around those thirty operations. An LLM produces syntactically correct PySpark from a SQL or Pandas description almost every time.

Pretending otherwise helps nobody. The syntax is not where your value is.

## Where your value moved to

Generated code is **usually correct and frequently expensive**. It optimises for "produces the right answer," not "produces it without shuffling 400 GB."

Your job is the review. That requires the mental model in Sections 05–07 — which is exactly why we spent an hour on them.

| CHECK | WHAT YOU ARE LOOKING FOR | THE TELL |
|---|---|---|
| **Python UDFs** | A loop or `@udf` doing something a built-in already does | `BatchEvalPython` in `.explain()` |
| **Shuffle count** | Aggregations or joins that could be reordered, filtered earlier, or broadcast | More stages than you expected in the UI |
| **Filter placement** | Filters applied *after* a join or aggregation instead of before | Empty `PartitionFilters` |
| **collect / toPandas** | Anything pulling a dataset rather than a result into the driver | The job dies on the driver, not an executor |
| **Unpartitioned windows** | `Window.orderBy()` with no `partitionBy` | One task running while everything else waits |
| **Write shape** | Missing `coalesce`; partitioning on a high-cardinality column | Thousands of tiny files in the output prefix |

CLOSE

# Six things to walk out with

### 1 · It's all open source

Spark came from Berkeley, lives at Apache, and runs on OpenJDK. Glue and EMR are wrappers and billing models, not the engine.

### 2 · Python is a remote control

Py4J sends a plan; the JVM does the work. That is why native DataFrame code runs at Scala speed — and why one Python UDF breaks the deal.

### 3 · Same Spark, different bill

Glue, EMR Serverless and EMR on EC2 run identical code. You are choosing an ops burden and a price curve, not a capability.

### 4 · Thirty functions

Eight families. Learn them properly and you can build almost anything asked of you. The other 570 are a lookup when the moment comes.

### 5 · Shrink, then route

Map-side combine turns a terabyte into megabytes. Hash routing makes one machine's local answer the global one. That is the whole algorithm.

### 6 · Your value is the review

Anyone can generate the syntax. Knowing that line 40 will shuffle 400 GB, skew onto one core, or cost $900 is the job.

If you know a data warehouse, if you know OLAP, if you know window functions — you walked in already holding 90% of this. Today was mostly spelling, plus the four places where the distributed reality bites.

NEXT

# Where you go from here

## Do this week

- Run `.explain(True)` on one job you already own and read the physical plan end to end
- Find your `shuffle.partitions` setting and compare it to your core count
- Search your codebase for `@udf` and `.collect()`
- Open the Spark UI on a real job and find the max-vs-median task time

## Read next

- Spark SQL **Performance Tuning** docs — short, and the highest value per page
- Apache Iceberg **spec** — the metadata section, 20 minutes well spent
- AWS **EMR Best Practices Guide**, and the Glue developer guide
- *Spark: The Definitive Guide*, chapters 15–19 for execution internals
- The **pyspark.sql.functions** API page — skim all 400 once. Just once.

## Habits

- Set `session.timeZone = UTC` in every job
- Partition filter first, always
- Search the built-ins before writing a UDF
- `overwritePartitions()` so reruns are idempotent
- Look at the bill weekly — cost is a performance metric

**Module 01 gave you the map. Module 02 gave you the vehicle.**

The labs are where you find out which parts you only *think* you understood. That is the point of them.

QUESTIONS

# The question you think is too obvious is the one to ask.